

# Lecture 15

## Bootstrapping GSW and TFHE

SEC 595: Encrypted Computing

## Roadmap

Lecture 14 gave bootstrapping for an arbitrary scheme: evaluate the decryption circuit under encryption, using an encryption of the secret key. This lecture carries it out for GSW (Lecture 13). Section 1 states what has to be shown. Section 2 writes GSW decryption as a Boolean circuit and measures its depth. Section 3 gives the evaluation key and the assumption it needs. Section 4 traces the noise through one reryption and derives the condition under which bootstrapping works. Section 5 shows how TFHE turns the same idea into a scheme that bootstraps after every gate.

## 1. The Idea

Bootstrapping changes one argument of Eval:

$$\Pi \mapsto C_{\text{Dec}},$$

the scheme's own decryption circuit. The algorithm, the keys and the ciphertext format are unchanged. Because the secret key enters  $C_{\text{Dec}}$  encrypted, the output is not the message but a *fresh encryption* of it.

Two things must be shown for GSW:

1.  $C_{\text{Dec}}$  is a circuit GSW can evaluate, with budget to spare (Sections 2 and 4);
2. the evaluator holds encryptions of the secret key (Section 3).

## 2. GSW Decryption as a Circuit

### 2.1 One Row Suffices

Recall from Lecture 13 that the key is  $s = (-\hat{s}, 1)^\top \in \mathbb{Z}_q^{n+1}$ , that  $C \in \mathbb{Z}_q^{\ell \times (n+1)}$  with  $\ell = (n+1) \log q$ , and that

$$C \cdot s = M \cdot Gs + e, \quad \|e\|_\infty < q/4.$$

The last row of  $G$  is  $(0, \dots, 0, q/2)$  and  $s_{n+1} = 1$ , so  $(Gs)_\ell = q/2$  and

$$(Cs)_\ell = \sum_{j=1}^{n+1} C_{\ell j} s_j = M \cdot \frac{q}{2} + e_\ell.$$

$M$  is recovered by asking whether this value is nearer 0 or  $q/2$ , which is Regev decryption (Lecture 7). Equivalently,  $M$  is the top bit of  $(Cs)_\ell + q/4 \bmod q$ : for  $M = 0$  the shifted value lies in  $(0, q/2)$ , for  $M = 1$  in  $(q/2, q)$ .

**Proposition 1.**  $C_{\text{Dec}}$  consists of one inner product of length  $n + 1$  followed by reading one bit.

The other  $\ell - 1$  rows of  $C$ s are not needed. This shrinks the circuit by a factor of  $\ell$  and does not change its depth.

## 2.2 The Inner Product over Bits

Choose  $\hat{s} \in \{0, 1\}^n$  binary (Lecture 6). Then  $C_{\ell j} s_j = (-C_{\ell j} \bmod q) \hat{s}_j$  for  $j \leq n$ , and the last term  $C_{\ell, n+1}$  is a constant. Write  $c_j$  for these constants, with  $q/4$  added to  $c_{n+1}$ . The inner product becomes:

- *Selection.*  $c_j \hat{s}_j$ : the  $\log q$  known bits of  $c_j$ , each ANDed with the key bit  $\hat{s}_j$ . One layer of AND gates.
- *Adder tree.*  $\sum_j c_j \hat{s}_j \bmod q$ : an addition of  $n + 1$  numbers of  $\log q$  bits, built from full adders. A full adder is

$$\text{sum} = (a \oplus b) \oplus c, \quad \text{carry} = ab \oplus c(a \oplus b),$$

two AND and three XOR gates, AND-depth 1.

- *Test.* Read the top bit of the result.

The constants  $c_j$  come from  $C$ , which the evaluator holds in the clear: they are hard-wired into the circuit, not encrypted inputs. An AND gate with one known input is either a wire (bit 1) or the constant 0 (bit 0), so the selection layer costs nothing.

## 2.3 GSW Supplies the Gates

From Lecture 13:

| Ciphertext operation          | On messages      | Gate |
|-------------------------------|------------------|------|
| $G^{-1}(C_1) C_2$             | $M_1 M_2$        | AND  |
| $C_1 + C_2 - 2G^{-1}(C_1)C_2$ | $M_1 \oplus M_2$ | XOR  |
| $G - G^{-1}(C_1)C_2$          | $1 - M_1 M_2$    | NAND |

Plain  $C_1 + C_2$  gives  $M_1 + M_2$ , which is XOR only when  $M_1 M_2 = 0$ . NAND alone is universal, so every Boolean circuit, in particular  $C_{\text{Dec}}$ , can be evaluated, subject to the noise budget.

## 2.4 Cost

**Proposition 2** (Cost of  $C_{\text{Dec}}$ ).  $C_{\text{Dec}}$  has on the order of  $(n + 1) \log q$  full adders and AND-depth

$$d = O(\log n + \log \log q).$$

*Proof sketch.* A layer of carry-save adders turns every three addends into two, at AND-depth 1; reducing  $n + 1$  addends to two takes  $O(\log n)$  layers and about  $n - 1$  rows of  $\log q$  full adders. A prefix adder adds the last two  $\log q$ -bit numbers in depth  $O(\log \log q)$ .  $\square$

Raising  $q$  buys a budget that grows like  $q$  (Section 4), while  $d$  grows only like  $\log \log q$ . The gap between what decryption costs and what the scheme can afford widens. This is why GSW is bootstrappable.

**Example 1** (The decryption circuit:  $n = 3$ ,  $q = 8$ ). The key bits are  $\hat{s}_1, \hat{s}_2, \hat{s}_3$ , with  $s_4 = 1$ , and there are four constants  $c_1, \dots, c_4$  of 3 bits each. Write  $W, X, Y, Z = (z_0, z_1, z_2)$  for the four selected addends  $c_j \hat{s}_j$  (with  $\hat{s}_4 = 1$ ), and subscripts for bits.

*Selection layer.*  $4 \times 3 = 12$  AND gates, each with a known input: free.

*Adder tree.* We need only bit 2 of  $W + X + Y + Z \bmod 8$ ; carries out of bit 2 are discarded.

1. Carry-save  $W, X, Y$ : full adders at bits 0 and 1, and a 3-input XOR at bit 2. Output  $S$  and carries  $K = (0, k_1, k_2)$ : 4 AND, 8 XOR.
2. Carry-save  $S, K, Z$ : a half adder at bit 0 (1 AND, 1 XOR), a full adder at bit 1, a 3-input XOR at bit 2. Output  $T$  and  $U = (0, u_1, u_2)$ : 3 AND, 6 XOR. The sum bit  $t_0$  is not needed, saving 1 XOR.
3. Top bit of  $T + U$ :  $t_2 \oplus u_2 \oplus (t_1 \wedge u_1)$ : 1 AND, 2 XOR.

Total:  $4 + 3 + 1 = 8$  AND and  $8 + 5 + 2 = 15$  XOR gates. The carries  $k_1, k_2$  have AND-depth 1;  $t_1 = s_1 \oplus k_1 \oplus z_1$  and  $u_1 = s_0 \wedge z_0$  have depth 1;  $u_2$ , whose carry ANDs  $z_1$  with  $s_1 \oplus k_1$ , has depth 2; and  $t_1 \wedge u_1$  has depth 2. The AND-depth is 2 (3 if the selection layer is counted). Only the adder tree adds depth.

*An instance.* Let  $\hat{s} = (1, 0, 1)$  and  $C_\ell = (3, 6, 7, 7)$ . Then  $(Cs)_\ell = 7 - 3 - 7 = -3 \equiv 5 = 4 + 1$ , so  $M = 1$  with  $e_\ell = 1$ . The constants are  $c = (-3, -6, -7, 7 + 2) \bmod 8 = (5, 2, 1, 1)$ , the selected addends are  $(5, 0, 1, 1)$ , their sum is  $7 = 111_2$ , and the top bit is  $1 = M$ .  $\checkmark$

### 3. The Evaluation Key

#### 3.1 A List of Ciphertexts

**Definition 1** (Bootstrapping key).

$$ek = (\text{Enc}(s, \hat{s}_1), \dots, \text{Enc}(s, \hat{s}_n), \text{Enc}(s, s_{n+1})),$$

one GSW ciphertext per key bit, under that same key;  $n + 1$  entries in all.

( $s_{n+1} = 1$  is public, so the last entry may be a trivial encryption.) Refreshing  $C$  means

$$\text{Eval}(ek, C_{\text{Dec}_C}, ek) = \tilde{C} \neq C, \quad \text{Dec}(s, \tilde{C}) = \text{Dec}(s, C).$$

The subscript records that  $C$  is hard-wired as constants; only the key bits are encrypted inputs. Constants cost no gates and no noise.

The inequality  $\tilde{C} \neq C$  is the point: the message is the same, but the noise of  $C$  has been consumed by the decryption test, whose answer is a bit.  $\tilde{C}$  carries only the noise of one evaluation of  $C_{\text{Dec}}$ .

#### 3.2 Circular Security

Publishing  $\text{Enc}(s, \hat{s}_i)$  is not covered by IND-CPA (Lecture 8, §1.5): in that game the messages are chosen without knowledge of the key.

**Definition 2** (Circular security). A scheme is **circular secure** if it remains secure when the adversary also holds encryptions of the secret key under that key.

No reduction from LWE to circular security of GSW is known, and no attack is known. It is an extra assumption. The alternative — a chain of keys, each encrypted under the next — has no cycle but fixes the number of bootstraps, so the scheme becomes *levelled*.

**Example 2** (What the key costs:  $n = 500$ ,  $q = 2^{60}$ ). (a)  $\ell = (n + 1) \log q = 501 \cdot 60 = 30\,060$ . One ciphertext has  $\ell \times (n + 1) = 30\,060 \times 501 = 15\,060\,060$  entries of 60 bits, that is 903 603 600 bits  $\approx 113$  MB.

(b) With a binary secret,  $ek$  holds  $n + 1 = 501$  ciphertexts:  $501 \times 113$  MB  $\approx 56.6$  GB.

(c) With a uniform secret encrypted bit by bit,  $ek$  holds  $(n + 1) \log q = \ell = 30\,060$  ciphertexts, about 3.4 TB. The decryption circuit must also rebuild each  $s_j$  from its bits, which is a multiplication rather than a selection.

These sizes are why plain GSW is a conceptual scheme; its descendants (FHEW, TFHE) keep its algebra but replace the matrices by ring elements.

## 4. Noise

### 4.1 The Accumulator

GSW multiplication is asymmetric (Lecture 13):

$$G^{-1}(C_1)C_2 : \quad \|e_{\text{new}}\|_{\infty} \leq \|e_1\|_{\infty} + \ell \|e_2\|_{\infty}.$$

Only the noise of the *second* operand is multiplied by  $\ell$ . So evaluate  $C_{\text{Dec}}$  **sequentially**:

- the running value (the *accumulator*) always in the decomposed slot  $C_1$ ;
- a fresh ciphertext from  $ek$  always in  $C_2$ .

Each gate then adds at most  $\ell E$  to the accumulator; nothing is multiplied.

### 4.2 Tracing One Recryption

**Proposition 3** (Bootstrapping condition). *Let each ciphertext in  $ek$  have  $\|e\|_{\infty} \leq E$ , and evaluate  $C_{\text{Dec}}$  as  $g$  gates with the accumulator in the  $G^{-1}$  slot. Then the noise  $N_i$  after  $i$  gates satisfies*

$$N_0 = E, \quad N_i \leq N_{i-1} + \ell E, \quad \text{so} \quad \|e_{\tilde{C}}\|_{\infty} \leq E(1 + g\ell),$$

and bootstrapping is correct when

$$E(1 + g\ell) < \frac{q}{4}.$$

*Proof.* Apply the bound of Section 4.1 with  $\|e_1\|_{\infty} = N_{i-1}$  and  $\|e_2\|_{\infty} \leq E$ , and sum the recurrence over  $g$  steps. Correctness of  $\tilde{C}$  then follows from  $\|e\|_{\infty} < q/4$  (Section 2.1).  $\square$

The noise of the input  $C$  does not appear in the bound. A NAND gate,  $G - G^{-1}(C_1)C_2$ , has the same noise as AND up to sign. The corrected XOR has noise  $(1 - 2M_2)e_1 + e_2 - 2G^{-1}(C_1)e_2$ , which adds at most  $(2\ell + 1)E$ ; counting XOR gates this way changes the bound by at most a factor of 3.

*Remark* (Balanced tree). If both operands of each AND are evaluated ciphertexts with noise  $\leq N$ , the result has noise  $\leq N + \ell N = (\ell + 1)N$ . A balanced tree of AND-depth  $d$  therefore gives  $E(\ell + 1)^d$ , exponential in the depth.

**Example 3** (The budget:  $n = 500$ ,  $E = 2^{10}$ ). (a)  $q = 2^{60}$ :  $\ell = 30\,060$  and  $q/4 = 2^{58} = 288\,230\,376\,151\,711\,744$ . The condition  $1024(1 + 30\,060g) < 2^{58}$  holds for  $g \leq 9\,363\,771\,680$ : at this  $g$  the left side is  $288\,230\,376\,141\,620\,224 < 2^{58}$ , and at  $g + 1$  it is  $288\,230\,376\,172\,401\,664 > 2^{58}$ .

(b)  $C_{\text{Dec}}$  has roughly 30 000 gates. The budget exceeds this by a factor of  $9\,363\,771\,680/30\,000 \approx 3.1 \times 10^5$ , so GSW is bootstrappable at these parameters, and hence fully homomorphic (under circular security).

(c)  $q = 2^{30}$ :  $\ell = 501 \cdot 30 = 15\,030$  and  $q/4 = 2^{28} = 268\,435\,456$ . Now  $1024(1 + 15\,030 \cdot 17) = 261\,643\,264 < 2^{28}$  but  $1024(1 + 15\,030 \cdot 18) = 277\,033\,984 > 2^{28}$ , so  $g \leq 17$ . Seventeen gates is not a decryption circuit: bootstrapping fails.

(d)  $q = 2^{60}$ , balanced tree:  $1024 \cdot 30\,061^3 = 27\,816\,995\,959\,788\,544 < 2^{58}$  but  $1024 \cdot 30\,061^4 \approx 8.4 \times 10^{20} > 2^{58}$ , so  $d \leq 3$ . Sequential accumulation is what makes bootstrapping possible.

## 5. TFHE: Bootstrapping After Every Gate

GSW bootstrapping is expensive, so a GSW-based scheme bootstraps rarely and spends its noise budget in between, as BFV, BGV and CKKS do (Lectures 10–12). TFHE (Chillotti, Gama, Georgieva and Izabachène) turns this around: it keeps ciphertexts small, makes bootstrapping fast, and bootstraps after *every* gate. There is then no depth limit and no levels to manage.

### 5.1 Ciphertexts and Gates

TFHE works over the torus  $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ , the reals modulo 1; implementations store a torus element as an integer modulo  $q = 2^{32}$  or  $2^{64}$ .

**Definition 3** (LWE ciphertext over the torus). With a binary key  $s \in \{0, 1\}^n$ , an encryption of  $\mu \in \mathbb{T}$  is  $(a, b) \in \mathbb{T}^{n+1}$  with  $a$  uniform and  $b = \langle a, s \rangle + \mu + e$  for a small error  $e$ . The **phase** is  $\varphi = b - \langle a, s \rangle = \mu + e$ .

A bit  $m$  is encoded as  $\mu = +\frac{1}{8}$  if  $m = 1$  and  $\mu = -\frac{1}{8}$  if  $m = 0$ . Adding ciphertexts adds phases. A gate is a public linear combination followed by one bootstrapping, which reads only the *sign* of the phase:

**Proposition 4** (NAND by one linear combination). *For bit encodings  $c_1, c_2$ , the ciphertext  $c = (0, \frac{1}{8}) - c_1 - c_2$  has phase  $\frac{3}{8}, \frac{1}{8}, \frac{1}{8}$  or  $-\frac{1}{8}$  for  $(m_1, m_2) = (0, 0), (0, 1), (1, 0), (1, 1)$ , plus the sum of the errors. If that sum is below  $\frac{1}{8}$  in absolute value, the phase lies in  $(0, \frac{1}{2})$  exactly when  $\text{NAND}(m_1, m_2) = 1$ .*

*Proof.*  $\frac{1}{8} - \mu_1 - \mu_2$  with  $\mu_i = \pm\frac{1}{8}$  gives the four values; each is at distance at least  $\frac{1}{8}$  from the boundaries 0 and  $\frac{1}{2}$ .  $\square$

Bootstrapping  $c$  outputs a fresh encryption of  $+\frac{1}{8}$  if the phase is in  $(0, \frac{1}{2})$  and of  $-\frac{1}{8}$  otherwise. NAND is universal, so any Boolean circuit can be evaluated gate by gate, each gate leaving a fresh ciphertext.

### 5.2 Blind Rotation

Bootstrapping must compute the sign of  $\varphi$  without knowing  $s$ . TFHE does it with a polynomial in  $\mathbb{T}[X]/(X^N + 1)$  (Lecture 3), using that multiplying by  $X^k$  is a negacyclic rotation.

1. *Rescale.* Round  $a$  and  $b$  to integers modulo  $2N$ :  $\tilde{a}_i = \lfloor 2Na_i \rfloor$ ,  $\tilde{b} = \lfloor 2Nb \rfloor$ , so that  $\tilde{\varphi} = \tilde{b} - \sum_i \tilde{a}_i s_i \bmod 2N$  approximates  $2N\varphi$ .

2. *Test polynomial.* Let  $v = \frac{1}{8}(1 + X + \dots + X^{N-1})$ . Since  $X^N = -1$ , the constant coefficient of  $X^{-k}v$  is  $+\frac{1}{8}$  for  $0 \leq k < N$  and  $-\frac{1}{8}$  for  $N \leq k < 2N$ : it is the sign of the phase.
3. *Rotate under encryption.* Start from the accumulator  $\text{ACC} = X^{-\tilde{b}}v$  (as a trivial ring-LWE ciphertext) and, for  $i = 1, \dots, n$ , set

$$\text{ACC} \leftarrow \text{CMux}(\text{BK}_i, X^{\tilde{a}_i} \text{ACC}, \text{ACC}),$$

where  $\text{BK}_i$  is a GSW-type (GGSW) encryption of  $s_i$  and  $\text{CMux}(C, d_1, d_0) = C \boxdot (d_1 - d_0) + d_0$  selects  $d_1$  when  $C$  encrypts 1. After  $n$  steps  $\text{ACC}$  encrypts  $X^{-\tilde{\varphi}}v$ .

4. *Extract and switch keys.* The constant coefficient of  $\text{ACC}$  is read off as an LWE ciphertext under a key of length  $N$ , and a key switch returns it to the key  $s$  of length  $n$ .

The product  $C \boxdot d$  of a GGSW and a ring-LWE ciphertext is GSW's multiplication: a gadget decomposition (Lecture 4) of  $d$  times  $C$ . Each  $\text{CMux}$  puts the accumulator in the decomposed slot, so its noise grows additively, as in the accumulator of Section 4.1. This is what keeps bootstrapping cheap.

### 5.3 Programmable Bootstrapping

Replacing the coefficients of  $v$  with the values of a function  $f$  makes bootstrapping return an encryption of  $f$  applied to the (rounded) phase. One bootstrapping thus refreshes the noise *and* evaluates a lookup table on a message of a few bits. This is how TFHE handles comparisons, ReLU and other non-polynomial functions that are costly in BFV, BGV and CKKS.

**Example 4** (One NAND gate, bootstrapped with  $N = 4$ ). Take  $m_1 = 1$  with error 0.01 and  $m_2 = 0$  with error  $-0.02$ , so the phases are  $\frac{1}{8} + 0.01 = 0.135$  and  $-\frac{1}{8} - 0.02 = -0.145$ . The combination  $(0, \frac{1}{8}) - c_1 - c_2$  has phase  $0.125 - 0.135 + 0.145 = 0.135$ . With  $2N = 8$ ,  $\tilde{\varphi} = \lfloor 8 \cdot 0.135 \rfloor = \lfloor 1.08 \rfloor = 1$ , which is in  $[0, 4)$ , so blind rotation returns  $+\frac{1}{8}$ : the output bit is  $1 = \text{NAND}(1, 0)$ . ✓

For  $m_1 = m_2 = 1$  with errors 0.01 and  $-0.02$ , the phase is  $0.125 - 0.135 - 0.105 = -0.115$ , and  $\tilde{\varphi} = \lfloor -0.92 \rfloor = -1 \equiv 7 \pmod{8}$ , which is in  $[4, 8)$ : the output is  $-\frac{1}{8}$ , bit 0. ✓

### 5.4 How TFHE Differs

|                | <b>BFV / BGV / CKKS</b>                       | <b>GSW</b>            | <b>TFHE</b>                                      |
|----------------|-----------------------------------------------|-----------------------|--------------------------------------------------|
| Plaintext      | vectors of integers or reals, packed in slots | a bit                 | a bit or a few-bit integer                       |
| Ciphertext     | pair of ring elements                         | matrix                | LWE vector                                       |
| Multiplication | tensor product and relinearization            | $G^{-1}(C_1)C_2$      | a gate: linear combination plus bootstrapping    |
| Noise          | levels, consumed until bootstrapping          | asymmetric growth     | reset after every gate                           |
| Bootstrapping  | rare and expensive                            | expensive             | every gate, fast                                 |
| Suited to      | arithmetic on many values at once             | theory; basis of TFHE | comparisons, branching, non-polynomial functions |

The trade is throughput for flexibility: BFV, BGV and CKKS process thousands of slots per operation, while TFHE processes one bit or small integer per bootstrapping but can evaluate any

function with no depth to plan.

## Summary

1. Bootstrapping GSW evaluates  $C_{\text{Dec}}$  in place of  $\Pi$ ; the output  $\tilde{C}$  decrypts to the same bit with fresh noise.
2.  $C_{\text{Dec}}$  is one inner product  $\sum_j C_{\ell j} s_j$  and a top-bit test. With a binary key it is one AND layer plus an adder tree, of depth  $O(\log n + \log \log q)$ .
3.  $ek$  holds an encryption of each of the  $n+1$  key bits under that key; this needs circular security.
4. Sequential accumulation exploits GSW's asymmetric noise:  $\|e_{\tilde{C}}\|_{\infty} \leq E(1 + g\ell)$ , independent of the input's noise. A balanced tree gives  $E(\ell + 1)^d$ .
5. Bootstrapping works when  $E(1 + g\ell) < q/4$ : true with a margin of about  $3 \times 10^5$  at  $q = 2^{60}$ , false at  $q = 2^{30}$ .
6. TFHE encrypts one bit (or a few) as an LWE vector and bootstraps after every gate by blind rotation, a chain of GSW-type CMux steps; with a programmable test polynomial the same step evaluates a lookup table.

## Exercises

1. In Example 1, take  $\hat{s} = (0, 1, 1)$  and  $C_{\ell} = (3, 6, 7, 7)$ . Compute  $(Cs)_{\ell}$ , the constants  $c_j$ , the selected addends and the output bit. Is this a valid encryption with  $\|e\|_{\infty} < q/4$ ?
2. Explain why the offset  $q/4$  is needed in Section 2.1. Give a value of  $(Cs)_{\ell}$  with  $q = 8$ ,  $M = 0$  and  $|e_{\ell}| = 1$  whose top bit, without the offset, is wrong.
3. Show that  $G - G^{-1}(C_1)C_2$  encrypts  $1 - M_1M_2$ , and that its noise is the negative of the noise of  $G^{-1}(C_1)C_2$ .
4. Redo Example 2 for  $n = 1000$ ,  $q = 2^{60}$ .
5. In Example 3(c), find the largest balanced-tree depth  $d$  with  $E(\ell + 1)^d < q/4$ .
6. For  $n = 500$  and  $E = 2^{10}$ , find the smallest  $\log q$  for which  $E(1 + g\ell) < q/4$  holds with  $g = 30\,000$ .
7. Describe a scheme that is IND-CPA secure but completely insecure once an encryption of its own key is published. (Hint: modify Enc on one distinguished message.)
8. In TFHE, give the linear combination of  $c_1, c_2$  (and a trivial ciphertext) whose sign gives AND, and the one for OR. Check all four inputs as in Section 5.1.